

# Bike Rental and Ride-Sharing Platform



**Module:** SE1020 - Object Oriented Programming

**Technology Stack:** Java, Spring Boot, Thymeleaf, Bootstrap, File Handling

**Repository:** <https://github.com/ashifahmed-924/OOP-bike-rental-system>

**Student Name:** M.N.A.Ahmed

**Student ID:** IT22920386

**Date:** 2026-04-23

## Contents

|                                           |   |
|-------------------------------------------|---|
| Introduction .....                        | 3 |
| Objectives.....                           | 3 |
| Technologies Used .....                   | 3 |
| Main Features .....                       | 4 |
| Project Structure .....                   | 4 |
| Object-Oriented Programming Concepts..... | 5 |
| CRUD Operations .....                     | 6 |
| File Handling.....                        | 6 |
| Class Diagram .....                       | 7 |
| User Interfaces .....                     | 7 |
| Challenges Faced.....                     | 8 |
| Git Repository and Commit History .....   | 8 |

# Introduction

This project is a web-based Bike Rental and Ride-Sharing Platform developed for the SE1020 Object Oriented Programming module. The system allows users to register, log in, manage bike rental listings, request bike bookings, approve rental requests, and handle ride-sharing between users. The backend is implemented using Java and Spring Boot MVC, while Thymeleaf is used for server-side page rendering. Instead of a database, the application stores persistent data in text files.

## Objectives

- Apply object-oriented programming concepts such as encapsulation, inheritance, abstraction, and polymorphism.
- Implement CRUD operations for user management, bike management, and ride bookings.
- Develop a user-friendly interface for different system roles.
- Use file read/write operations for persistent storage.
- Track development using Git and GitHub.

## Technologies Used

### **Backend :**

- Java
- Spring Boot
- Spring MVC
- Thymeleaf
- Maven

### **Frontend :**

- HTML/ CSS
- Bootstrap
- Custom CSS theme

## Persistence and Tools:

- Text file storage
- Git
- GitHub

## Main Features

- Public home page with bike listing
- User registration and login
- Role-based dashboard redirection
- Admin user management: create, search, update, delete
- Operator bike management: add, view, update, delete bikes
- Operators can only edit/delete bikes they added
- Ride sharers can request to rent available bikes
- Operator must accept or decline rental requests
- Accepted rides become open for sharing
- Ride bookers/passengers can request to join shared rides
- Ride sharers can accept or decline passenger share requests
- Bike status automatically syncs between AVAILABLE and BOOKED
- File-based persistence using text files

## Project Structure

- src/main/java/app - Spring Boot application, controllers, and support classes
- src/main/java/model - domain models and user subclasses
- src/main/java/dao - file-based DAO classes
- src/main/resources/templates - Thymeleaf UI templates
- src/main/resources/static/assets - CSS and image assets
- src/main/webapp/data - persistent text files

## Current Roles

- ADMIN: manages platform users.
- OPERATOR: adds bikes, manages own bikes, and approves rental requests for bikes they own.
- RIDE\_SHARER: rents a bike, creates a route, and manages passenger share requests.
- RIDE\_BOOKER: searches ongoing shared rides and requests to join them.
- RIDER: fallback/legacy role that can both book bikes and join shared rides.

## Object-Oriented Programming Concepts

### Encapsulation:

Encapsulation is used in the model classes through controlled access to data fields using getters and setters. Examples include Bike, RideBooking, and ShareRideRequest.

RideBooking.getShareRequests() also returns a copy of the list to protect internal data.

### Inheritance:

Inheritance is used to create a hierarchical relationship between classes. Person is the parent class of User, and User is further extended by AdminUser, OperatorUser, RiderUser, RideSharerUser, and RideBookerUser. Controllers also inherit common helper methods from BaseWebController.

### Polymorphism:

Polymorphism is achieved through method overriding and dynamic object creation. Different user subclasses override getRole() and getDashboardPath() methods. UserFactory.createUser() returns different subclass objects using the parent User reference type. The toString() method is also overridden in User, Bike, and RideBooking.

## Abstraction

Abstraction is implemented using abstract classes such as Person and User. These classes define common attributes and behaviors while forcing subclasses to implement methods like `getRole()` and `getDashboardPath()`.

## CRUD Operations

| Module        | Create                           | Read                                 | Update                                                              | Delete                   |
|---------------|----------------------------------|--------------------------------------|---------------------------------------------------------------------|--------------------------|
| Users         | Registration / admin create user | Login, list users, search users      | Admin update user                                                   | Admin delete user        |
| Bikes         | Operator add bike                | Public bike list, operator bike list | Operator update own bike                                            | Operator delete own bike |
| Ride Bookings | Ride sharer requests bike rental | Rider/operator booking pages         | Edit booking, approve/decline rental, approve/decline share request | Cancel/delete booking    |

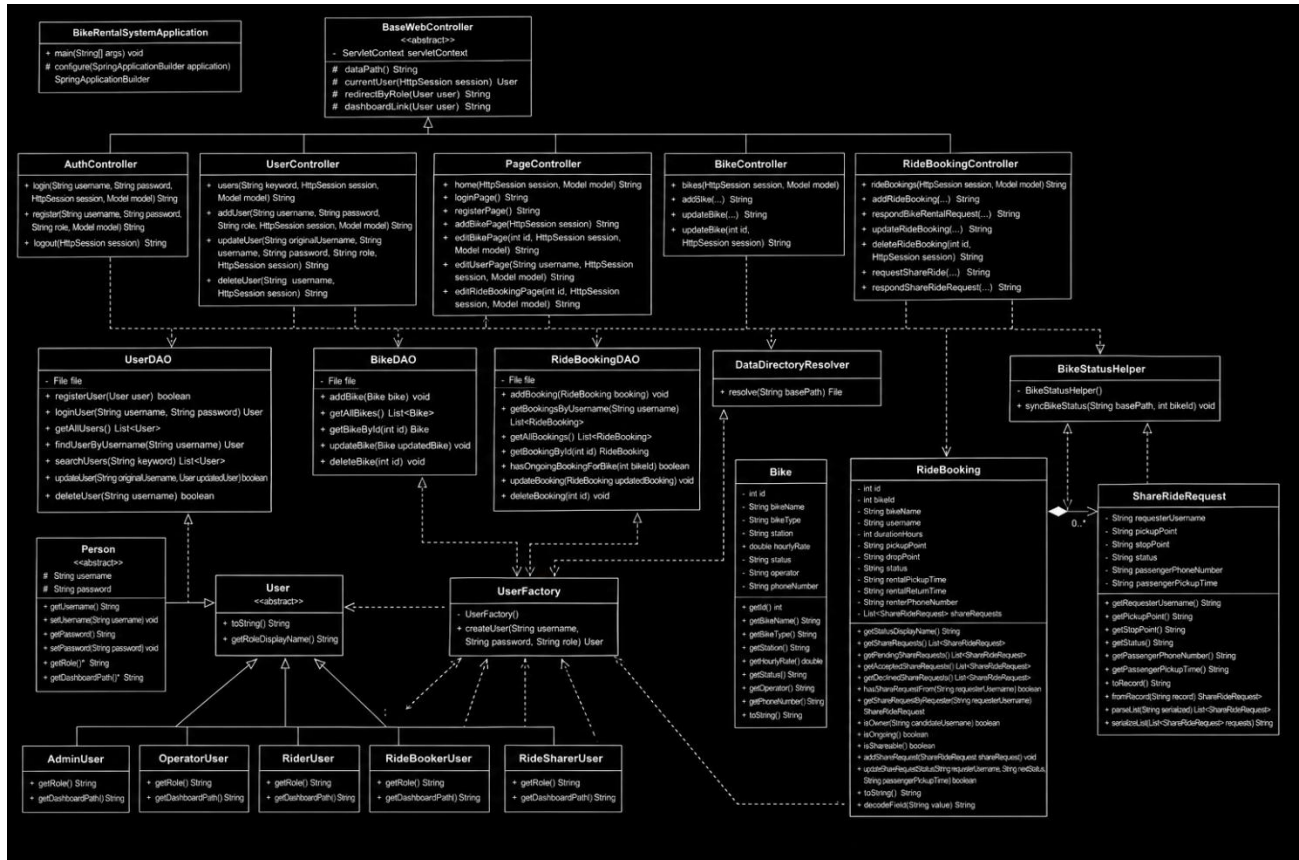
## File Handling

The application stores data using text files instead of a relational database.

- users.txt: username:password:role
- bikes.txt: id:bikeName:bikeType:station:hourlyRate:status:operator:phoneNumber
- ride-bookings.txt: booking data plus encoded ride-sharing request data

File operations are handled through UserDAO, BikeDAO, DataDirectoryResolver and RideBookingDAO.

# Class Diagram



High Quality Image:

[https://drive.google.com/file/d/10F\\_ytcnNzeZal3kLzfpnftwZOsQl9ZQ/view?usp=sharing](https://drive.google.com/file/d/10F_ytcnNzeZal3kLzfpnftwZOsQl9ZQ/view?usp=sharing)

## User Interfaces

- Home page
- Login and registration pages
- Registration page
- Admin user management page
- Edit user page
- Operator bike management page

- Add bike page
- Edit bike page
- Ride bookings / ride-sharing page
- Edit ride booking page

## Challenges Faced

- Migrating the project from JSP/Servlet architecture to Spring Boot.
- Converting JSP pages into Thymeleaf templates.
- Preserving file-based data storage after the framework migration.
- Managing persistent data using text files instead of a database.
- Keeping bike availability status synchronized with booking status.
- Storing ride-sharing requests inside booking records using file serialization.
- Designing role-based access for Admin, Operator, Ride Sharer, and Ride Booker users.

## Git Repository and Commit History

**Repository URL:** <https://github.com/ashifahmed-924/OOP-bike-rental-system>

Version control for the project was managed using Git and GitHub. The commit history shows the progress of the application from the initial implementation to the final Spring Boot and Thymeleaf migration.

| Commit ID         | Message                                      |
|-------------------|----------------------------------------------|
| 8b34f60           | Small UI fixes and visual cleanup            |
| 2a9bcbf           | Enhanced frontend styling and responsiveness |
| 2a0455c           | Add OOP concept explanation comments         |
| f91097d           | Minor UI spacing and alignment improvements  |
| 91aed09 / 3e59ead | Improve bike rental role workflow            |
| dd0acea           | Add project report documents                 |



|         |                                          |
|---------|------------------------------------------|
| 03b09ee | Restructured: Project folders            |
| 681ffa4 | Convert app to Spring Boot and Thymeleaf |
| 10d8d84 | first commit                             |